Aufbau eines Labs zum Thema NFC-RFID Authentifizierung



Datum: 12.04.2026

Dozent: Dr. Eskhita Radwan

Gruppe: 3a

Autoren:

Mazreku Dorian, Massaro Dario Leon, Paunovic Sascha,

Kemény Marc-André, Jund Johannes

1. Szenario

In diesem Lab wurde ein NFC-basiertes Zutrittssystem prototypisch umgesetzt und anschliessend sicherheitstechnisch bewertet. Das System soll beispielhaft eine typische Situation abbilden.

Fallbeispiel: Zugangskontrolle zu einem Raum (z.B. Labor/Serverraum).
Nutzer halten einen NFC-Tag an ein Lesegerät. Bei Erfolg wird „Zutritt erlaubt“ signalisiert, andernfalls „Zutritt verweigert“. Zusätzlich wird jeder Zugriff protokolliert.

Kernannahme des Prototyps: Die Entscheidung „Zutritt ja/nein“ basiert ausschliesslich auf der UID (Unique Identifier) des Tags. Diese UID wird ausgelesen und gegen eine lokale Whitelist geprüft. Der Ablauf wird visuell über LEDs sowie technisch über Logeinträge nachvollziehbar gemacht.

2. Darstellung der Komponenten

2.1 Lesegerät

- Raspberry Pi 5 als zentrale Steuereinheit
    - Betriebssystem: Debian 13 (Trixie)
- PN532 NFC-Modul zur NFC-Kommunikation im SPI-Modus
- Breadboard mit 2 LEDs:
    - Grün: Zugriff gewährt
    - Rot: Zugriff verweigert
- Applikationslogik in Python

2.2 Karte / Tag-Komponente

- MIFARE Classic 1K
    - Standard: ISO/IEC 14443A

2.3 Analysetools

- Flipper Zero

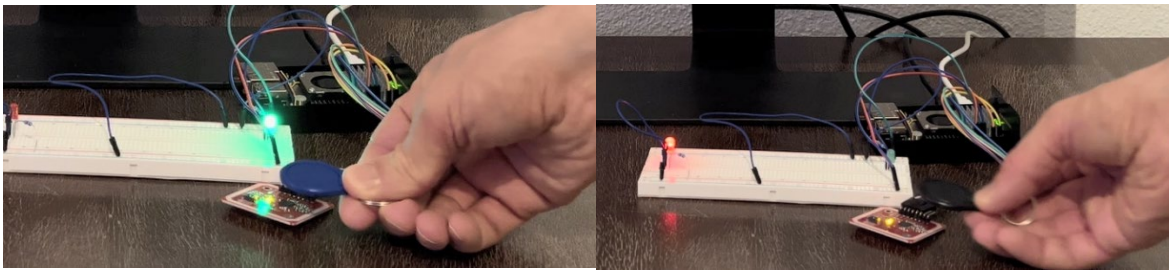
2.4 Backend (im Lab lokal realisiert)

Die „Backend-Funktion“ wird lokal auf dem Raspberry Pi umgesetzt:

- Whitelist: lokal gespeicherte Liste erlaubter UIDs
- Logfile: speichert Zeitstempel, UID, Ergebnis (allowed/denied)

In einem realen System wäre „Backend“ typischerweise ein zentraler Dienst (z.B. Datenbank + Auth-Service), im Lab ist es bewusst lokal gehalten, um die Sicherheitsanalyse auf den Kernmechanismus zu fokussieren.

2.5 Physischer Aufbau der Komponenten

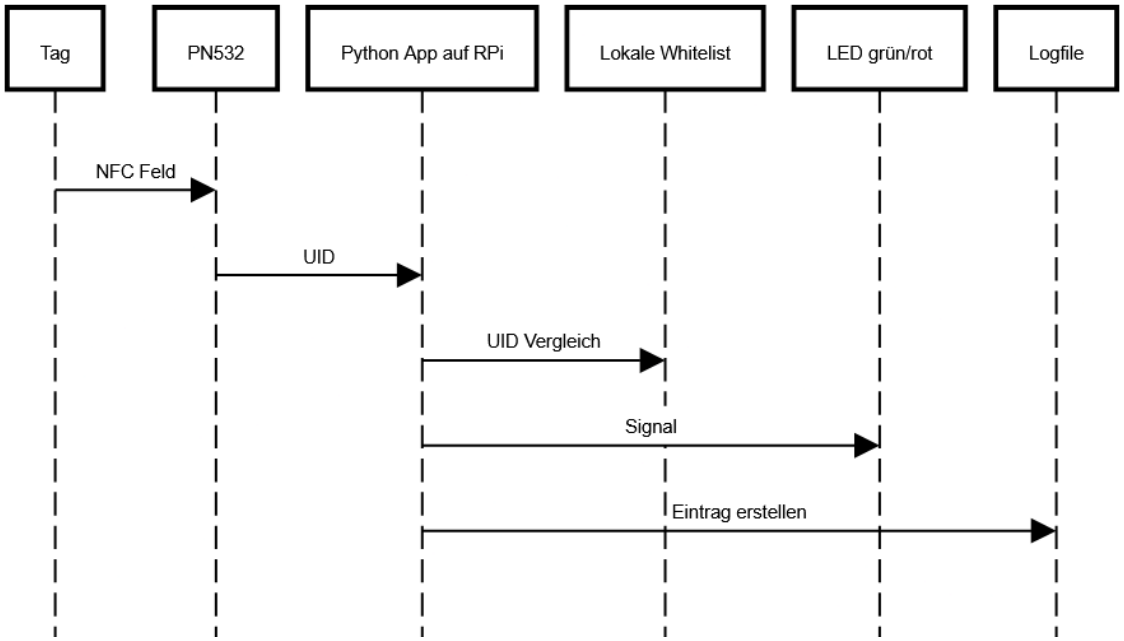


3. Beschreibung des Authentifizierungsablaufs

3.1 Ablauf im Normalbetrieb

1. Tag wird an PN532 gehalten
2. PN532 erkennt Tag und liest die UID
3. Python-App vergleicht UID mit Whitelist
4. Entscheidung:
    a. UID vorhanden → Zugriff gewährt → grüne LED
    b. UID nicht vorhanden → Zugriff verweigert → rote LED
5. Logging: Zeitstempel + UID + Resultat

3.2 Ablauf als Sequenzdiagramm



4. Lab Einrichtung

4.1 Physiche Einrichtung

1. Korrekte Verkabelung aller Komponenten (Pins, Breadboard, Strom)

4.2 Raspberry Pi

2. Einrichtung des Debian Linux Betriebssystem

3. Installation des Python Package: (sudo) apt install python –y

4.3 Python App

4. Python Skript auf dem momentanen Ordner kopieren (siehe Anhang)

5. Text-Datei “authorized_cards.txt” erstellen und bearbeiten: nano authorized_cards.txt

6. In der Text-Datei folgenden Eintrag hinzufügen: 9D C9 BC 02;NFC-Tag (blue)

7. Datei Speichern und Textbearbeitung beenden: CTRL + O und danach CTRL + X

8. Source für Python angeben: source .venv/bin/activate

9. Python-Applikation starten: python nfc_access_log.py

4.4 Log

10. Log File nach dem Testen auslesen: cat access_log.txt

5. Versuchsdurchführung

Das System wurde auf dem Raspberry Pi gestartet, und die Python-Applikation lief in einer virtuellen Umgebung. Parallel dazu wurde das Logfile in einem zweiten Terminal live überwacht, um alle Zugriffe in Echtzeit nachvollziehen zu können (vgl. Videoclips).

Im ersten Schritt wurde der Normalbetrieb getestet. Hält man einen autorisierten Tag (im Labor: blauer Tag) an das PN532-Modul, erkennt das System die UID, vergleicht sie mit der Whitelist und gewährt Zugriff. Dies wird durch die grüne LED signalisiert und im Logfile festgehalten.

Wird ein nicht autorisierter Tag (im Labor: schwarzer Tag) verwendet, findet sich dessen UID nicht in der Whitelist. Das System verweigert den Zugriff, die rote LED leuchtet auf, und der Vorgang wird im Logfile als verweigert protokolliert.

Anschliessend wurde ein Angriffsszenario durchgeführt. Der autorisierte Tag wurde mit dem Flipper Zero ausgelesen, und dessen UID über die Emulation erneut dargestellt. Der Flipper wurde ans PN532-Modul gehalten und simulierte den autorisierten Tag.

Das System konnte den Unterschied zwischen echtem und emuliertem Tag nicht erkennen und gewährte Zugriff, wie die grüne LED und der Logeintrag bestätigten. In einem weiteren Versuch wurde ein nicht autorisierter Tag (im Labor: schwarzer Tag) mit den Daten des autorisierten Tags beschrieben, also geklont. Auch dieser Tag wurde daraufhin als legitim erkannt und erhielt Zugriff. Für das System besteht kein Unterschied zwischen Original und Klon.

6. Analyse potenzieller Sicherheitsrisiken

Das aktuelle System basiert ausschliesslich auf der UID-Identifikation, was ein erhebliches Sicherheitsrisiko darstellt. Da die UID unverschlüsselt übertragen wird und mit Standardwerkzeugen auslesbar ist, fungiert sie nicht als Geheimnis, sondern lediglich als öffentliches Merkmal.

- **Cloning:** Da das System nicht zwischen Original und Kopie unterscheidet, kann eine autorisierte UID problemlos auf einen leeren Tag kopiert oder per Smartphone emuliert werden. Im Lab wurde bestätigt, dass das System diese Klone ohne Weiteres als legitime Nutzer akzeptiert. [1]
- **Replay:** Mangels zeitlicher Varianz oder Transaktionsbindung (z. B. Nonces) bleibt eine einmal abgefangene UID dauerhaft gültig. Angreifer können das Signal aufzeichnen und zu einem späteren Zeitpunkt wiederholen, ohne dass das System die fehlende „Frische“ der Daten bemerkt. [2]

7. Empfehlung geeigneter Schutzmassnahmen

Um das System abzusichern, muss der Wechsel von einer blossen Identifikation zu einer kryptographischen Authentisierung erfolgen:

- **Challenge-Response-Verfahren:** Anstatt die UID zu prüfen, sollte der Leser eine Zufallszahl (Nonce) senden, die der Tag mit einem geheimen Schlüssel signiert. Da sich die Challenge bei jedem Vorgang ändert, werden Replay-Angriffe wirkungslos. [4]
- **Sichere Hardware:** Der Einsatz veralteter Tags (wie MIFARE Classic) sollte durch moderne Standards mit **AES-Verschlüsselung** ersetzt werden. Diese schützen den geheimen Schlüssel vor dem Auslesen und verhindern so effektives Cloning. [1]
- **Schlüsselmanagement:** Ein robustes System erfordert diversifizierte Schlüssel (einzigartig pro Tag) sowie eine zentrale Verwaltung, um kompromittierte Identitäten sofort sperren zu können. [1]
- **Zusatzmassnahmen:** Begleitend sollten Rate-Limiting (Schutz gegen Brute-Force) und erweiterte Audit-Logs implementiert werden, um auffällige Zugriffsmuster frühzeitig zu erkennen. [1]

Erst die Kombination aus manipulationssicherer Hardware und einem dynamischen Verschlüsselungsprotokoll macht das System resistent gegen unbefugte Duplikation.

Kurzfazit

Das Lab zeigt eindeutig: Ein Zugriffssystem, das nur die UID prüft, ist nicht sicher, weil UID leicht gelesen, emuliert, geklont und damit auch beliebig replayed werden kann. Für reale Zutrittsszenarien ist eine kryptographische Authentisierung (Challenge-Response) mit sicheren Tags und sauberem Schlüsselmanagement zwingend erforderlich.

Quellen

[1] BLUEFIRE REDTEAM, RFID Cloning Explained, 2026, abgerufen am 11.04.2026
https://bluefire-redteam.com/rfid-cloning-explained/

[2] Wikipedia, Replay Attack, 2026, abgerufen am 11.04.2026
https://en.wikipedia.org/wiki/Replay_attack

[3] Wikipedia, Challenge-Response authentication, 2026, abgerufen am 11.04.2026
https://en.wikipedia.org/wiki/Challenge%E2%80%93response_authentication

Anhang

Abgabe

Inhalt des Abgabeordners:

- Python Skript (mithilfe von KI erstellt)
- Videoaufnahmen des Versuchs